

Deep Learning

Term Project Report

John Maguire 9081520250

Data

I was given a DTD of 47 classes and selected 20 classes of 2400 samples to train our model with. As I used a ResNet 18 model each image was resized to a standard shape of 3x224x224. I used a validation dataset and split the set 80% for training and 20% for validation approximately. This ensured a balanced class representation in both sets. For preprocessing I used normalisation using the ImageNet mean and standard deviation. The data split was only for train/val as the test data was unseen.

Model

For the model I fine-tuned a pretrained ResNet18 model using torchvision.models. ResNet is a relatively lightweight model with few parameters compared to deeper models. To match the 20 class requirements, I replaced the final fully connected layer by adding dropout and a linear layer. I followed a transfer learning strategy of Initially freezing all layers except the new FC layer. I then tested this against the results of unfreezing layer 4, layer 3 and even layer 2. Eventually I decided on only unfreezing layer 4 and the FC layer as this provided the best generalisation by only updating the high-level features which were most relevant to the task.

I found that with layer 4 unfrozen the model was overfitting and with it frozen the model was underfitting. However, after a lot of testing I reduced the overfitting by adding regularisation and augmentation methods.

Training

As discussed, I used various data augmentation and regularisation methods. To improve generalisation, I implemented the following transforms in utils.py

```
train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.RandomRotation(5),
    transforms.ToTensor(),
    normalize,
    transforms.RandomErasing(p=0.2, scale=(0.02, 0.2), ratio=(0.3, 3.3), value='random')
```

These transforms increase the diversity of the training images and therefore improve the model's ability to generalise unseen data. In early testing I only implemented normalisation and rotation, however as overfitting was a big problem for me, I must implement various random transforms such as horizontal flip, colour jitter and randomresizedcrop.

- In terms of regularisation techniques, I implemented mix-up $p=0.5$, $\alpha=0.8$ which combines two images and their corresponding labels. It was applied with 50% probability for each batch iteration.
- Dropout of 0.7 was used before the final FC layer to reduce overfitting. I used weight decay of $5e-3$ to penalise large weights.
- Label smoothing was initially present but during testing it was removed in favour of mix-up.
- For the optimiser I used Adam with a learning rate of $1e-4$ and weight decay $5e-4$
`optimizer = torch.optim.Adam(model.parameters(), lr=args.learning_rate, weight_decay=5e-3)`
- For learning rate scheduler I used CosineAnnealingLR to decrease learning rate
`scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args.epochs)`
- For the hyperparameters I chose 12 epochs as the validation set was steadying earlier
 - A slightly lower learning rate of $1e-4$ gave optimal results
 - A batch size of 32 was also optimal as a lower batch was faster but affected generalisation

Experiments

During testing the model and augmentations went through many iterations. Initially I trained a CNN model from scratch which turned out to be inefficient and thus I changed to a ResNet model. I used label smoothing at first but decided on using mix-up instead. With ResNET before weight decay and mix-up were implemented, I have results such as

Loss:0.3456
Train Accuracy: 89.17%
Validation Accuracy: 72.29%

Which indicated overfitting. This was the main problem with my experiments as the overfitting problem was hard to fix. After adding data augmentation such as Colour Jitter and Random Resized Crop aswell as mixup I had these results.

Train Loss: 0.6165
Train Accuracy: 87.19%
Validation Accuracy: 81.88%

This indicated good generalisation however the train loss was high which was worrying. To fix this I unfroze layer 4 of the ResNet model.

```
for name, param in self.backbone.named_parameters():
```

```
if any(layer in name for layer in ["layer4", "fc"]):  
    param.requires_grad = True
```

I then experimented with different values of weight decay and dropout to improve generalisation and prevent overfitting.

I found that a high dropout of .07 was needed to prevent overfitting and I also chose a high weight decay of $5e-3$. This is because ResNet 18 has a high capacity and can easily memorise the training data. Even with only layer 4 unfrozen overfitting was an issue. Therefore, I experimented with a variety of regularisation and augmentation methods. I experimented with Cutmix instead of Mixup but it gave a much larger loss and worse training accuracy after 12 epochs

Train Loss: 0.9931

Train Accuracy: 53.18%

Validation Accuracy: 80.21%

Therefore, I switched to Mixup and adjusted regularisation values to get my final results

Results

My final results are as follows

[Epoch 1/15] Train Loss: 2.5470 Train Accuracy: 27.71% Validation Accuracy: 69.17%	[Epoch 2/15] Train Loss: 1.2877 Train Accuracy: 62.19% Validation Accuracy: 77.29%
[Epoch 3/15] Train Loss: 0.9100 Train Accuracy: 72.81% Validation Accuracy: 82.08%	[Epoch 4/15] Train Loss: 0.6904 Train Accuracy: 78.96% Validation Accuracy: 81.46%
[Epoch 5/15] Train Loss: 0.5283 Train Accuracy: 85.36% Validation Accuracy: 84.17%	[Epoch 6/15] Train Loss: 0.4425 Train Accuracy: 86.98% Validation Accuracy: 84.38%

[Epoch 7/15] Train Loss: 0.3435 Train Accuracy: 90.94% Validation Accuracy: 85.21%	[Epoch 8/15] Train Loss: 0.3089 Train Accuracy: 91.82% Validation Accuracy: 82.71%
[Epoch 9/12] Train Loss: 0.2555 Train Accuracy: 94.38% Validation Accuracy: 82.71%	[Epoch 10/12] Train Loss: 0.2224 Train Accuracy: 95.36% Validation Accuracy: 83.96%
[Epoch 11/12] Train Loss: 0.2044 Train Accuracy: 95.42% Validation Accuracy: 83.12%	[Epoch 12/12] Train Loss: 0.1737 Train Accuracy: 96.77% Validation Accuracy: 82.71%

Analysis

The loss steadily decreases from 2.55 to .017 which indicates good convergence

The training accuracy is strong and reaches to 96%

However this indicates overfitting as from epoch 6 onwards the validation accuracy fluctuates. This is due to the small data set and ResNet learning too well. Even with good regularisation and data augmentation overfitting still occurs.